Usage

Keys

Key	Action
`→` `↓` `Space` `PageDown`	next slide (audience follows)
`←` `↑` `PageUp` `Backspace`	previous slide
`Home` / `End`	first / last
`Tab` / `Shift+Tab`	move the **preview only** — the audience does not follow
`Enter`	show the previewed slide
`Esc`	cancel the preview
`b` / `w`	blank black / blank white
`p` / `r`	start-pause / reset the timer
`s`	swap presenter and audience displays
`f`	toggle audience fullscreen
`o` / `F5`	open / reload
`d`	diagnostics bundle
`q`	quit

Presenter remotes usually emit `PageUp`/`PageDown`, media keys or browser back/forward — all bound by default. A remote whose keys the toolkit cannot name is still usable: press the key and the presenter window offers to bind it, storing the raw scancode in `settings.toml`.

In the layout designer: `Ctrl/Cmd+Z` and `Ctrl/Cmd+Shift+Z` undo and redo, `Ctrl/Cmd+S` saves, and with a divider focused the arrow keys move it 1% at a time (5% with `Shift`).

Starting the audience window

Only the presenter window opens initially. **Start** ▾ beside the hamburger is a split dropdown button: **Start** uses the saved audience display, while the arrow lists the connected displays so one click both selects a projector and starts the audience. It also offers a five-second delayed start (switch to the projector workspace during the count) and a windowed start for manual placement. The matching **Stop** button removes the audience window entirely.

On Niri, Pulpit uses compositor IPC, so choosing the projector sends the audience window to that output's active workspace. Other Wayland compositors still explain when they require manual placement.

Presenter layouts

The presenter screen is not hard-coded. It is a **layout**: a tree of splits and cells with a widget in each cell, rendered proportionally into whatever window it lands in. Four built-in layouts ship with the application, and any of them can be duplicated into an editable copy.

```
pulpit --layouts                        # the layout library
pulpit --edit-layout slide-next-notes   # straight into the designer
```

…or the **Layout: …** button in the presenter window.

The tree model lives in `crates/pulpit-app/src/layout` and each widget family in `crates/pulpit-app/src/widgets/<family>/`. Both are pure — no UI types, no clock, no rendering — so every structural rule is unit-tested without a display. The designer and the presenter view are two projections of the same values, which is what makes "what you designed is what you present" true rather than aspirational.

The tree

Two node types only:

- a **split** with two or more children in one direction, plus their relative sizes, a gap and a minimum child size;
- a **leaf** cell holding at most one widget, plus padding, background and what to do when it is empty. Neighbouring cells are separated by one muted hairline in the split gutter; cells and widgets do not draw perimeter borders.

The tree is kept **canonical**: a split never directly contains a child split of the same direction. That single rule is what makes divider behaviour predictable:

- Splitting a cell **in the same direction as its parent** inserts a divider into the parent. Splitting the middle of a three-across row left-and-right gives four across, not three-across-with-a-nested-pair.
- Splitting **perpendicular** always nests.
- The selected cell's space is halved; its siblings keep their sizes.
- Deleting a node gives its space back to its siblings in proportion. A split left with one child dissolves, and if the survivor now sits inside a split of its own direction it is flattened in.

Layouts are stored proportionally, so they scale to any screen without letterboxing or reflow. The editor's aspect-ratio selector changes only what the canvas previews — never the saved layout. When the presenter screen's real ratio is far from the design ratio, the presenter window shows one dismissible notice suggesting a review at that ratio.

Widgets

One directory per family — slides, notes, timing, navigation, status — each with a pure `model.rs` (configuration, defaults, bounds, capabilities, patches, the display decisions, and its own validation) and a `view.rs` that takes only the input facets it draws.

Every presentation property — variant, scale, alignment, and slide fit — is a constant in `widgets/tokens.rs`, not a per-widget setting. Colour comes only from the seven global roles in the active Light or Dark palette. Widgets size themselves to the cell they are given.

One widget is **compound** — `Previous + Current + Next` — and it counts as its constituents everywhere: placing it satisfies the current-slide requirement, and a bare `Current Slide` may then not be placed beside it. The timer, the clock and navigation are deliberately *not* compound: a layout may put the buttons along the bottom and the slider in a rail, or leave either out.

Single-instance widgets are exactly: Speaker Notes, Slide Buttons, Slide Slider, Pause or Resume, End Presentation, Annotations, Media Transport. Everything else may repeat. A single-instance widget already in the layout shows **Already in Layout** on its library card and cannot be dragged.

Media Transport

Play, pause and scrub whatever video or animation is on the slide the *audience* is seeing — never the slide the presenter has previewed ahead to, since pressing play must not start a clip nobody is watching.

It exists on the presenter's layout rather than inside the media because the two views consume the same overlay frames: a control drawn in the content is a control on the projector. The generated wrapper pages therefore draw nothing, and this widget reaches them through the media protocol instead.

What it offers follows what the content can answer. A clip gets a scrub bar and a mute button; an animation has no playhead and no audio, so it gets the button alone. Media whose runtime never started still gets a transport, drawn inert and reading **Not playing**.

Accent colours
Cyan (default), amber, white or slate — the sanctioned palette, no arbitrary colours. **Amber is reserved for timing displays.** The Timer takes it by default and other widgets are not offered it; choosing it elsewhere is refused with an explanation rather than silently accepted, so the timer stays the one thing on the screen that reads as urgent.

Editing

There is no properties panel. Everything is done on the canvas or from the toolbar, and the canvas draws the real widgets.

Action	How
Select a pane	click it
Add a pane	**Pane left / right / above / below** in the toolbar, beside the selection
Place a widget	drag a card from the library onto a pane, or onto a pane's edge to split and place in one gesture
Move a widget	drag it between panes
Clear a pane	right-click it, or ✕ in the toolbar
Remove a pane	the same again on an empty pane
Resize	drag a divider; grab it near a corner to move a whole line of aligned dividers; ←/→ moves a focused divider 1%, `Shift` 5%
Even split	double-click a divider
Undo / redo	↺ and ↻ in the toolbar, `Ctrl/Cmd+Z`, `Ctrl/Cmd+Shift+Z`
Save	`Ctrl/Cmd+S`, or **Save** when leaving

Four add-pane buttons replace the ready-made shapes: three across is **Pane right** twice, and where the new pane lands is always obvious from the button you pressed.

Dropping onto an occupied cell asks: **Replace Existing Widget**, **Swap Widgets** (between two cells) or **Cancel**. Nothing is destroyed in one press: removing a pane takes its widget first and the pane itself on the next press, and both steps are undoable. Removing a split takes only the panes on either side of it.

There is no save button and no discard button. Leaving the editor with unsaved changes asks **Save**, **Discard** or **Cancel**, which is the moment both questions mean something; a toolbar that can throw work away is a mis-click looking for somewhere to happen. Validation warnings are reported when you save rather than in a panel.

Undo covers every editing action — structure, resizes, placement — is unbounded within the session, and is *not* cleared by saving, so you can undo past a save. Closing the editor clears it.

Validation

Validation inspects **configuration, not presence**: a Slide Buttons widget with its forward button hidden does not satisfy the forward-navigation requirement.

Warnings — no current slide, no forward or backward control, notes too small, a widget that does not fit its cell, empty cells, an unreadable timer — are explained and then allowed. A presenter may deliberately want a layout with no notes. Only structural corruption and instance-limit violations block, which is what protects an import from a hand-edited file. Geometry warnings depend on the screen the layout is checked at, so the aspect-ratio selector changes what the checks say.

Files

Custom layouts live in `<config>/layouts/<id>.json`, written atomically. There is no import or export in the interface — the files themselves are the interchange format, and copying one into that directory is the whole of it. The shape, with a `format_version`:

```
{
  "format_version": 1,
  "name": "Conference Layout",
  "design_ratio": "sixteen-nine",
  "root": {
    "type": "leaf",
    "id": 0,
    "widget": {
      "kind": "timer",
      "style": { "variant": "standard", "scale": 1.0, "alignment": "center" },
      "config": { "timer": { "warning_minutes": 5 } }
    },
    "padding": 8.0,
    "background": "none",
    "border": "none",
    "empty_behavior": "show-blank-panel"
  }
}
```

`border` remains in format 1 files for compatibility, but is no longer rendered. New built-in layouts write `"none"`; the split gutter owns visual separation so adjacent cells cannot produce doubled edges or an outer frame.

A widget carries its kind, the style every widget has, and the configuration only its family can have. The two must agree: a file claiming a title holds notes options is refused rather than repaired.

On import Pulpit refuses a newer format version with a clear message, runs the full validation, renumbers node ids so they cannot collide, and appends a numeric suffix if the name is taken. An imported layout is always a custom layout, even if it was exported from a built-in. Four committed fixtures under `src/layout/fixtures/` pin this shape: they round-trip byte for byte and, between them, contain every widget kind.

Built-ins

Layout	For
Slide + Next + Notes	The slide-first default: current slide at 72% width, with next slide, notes and navigation in a rail.
Slide + Notes Beside	A 75/25 split that fits a 4:3 slide without padding on a 16:9 presenter display.

Slide + Time Below	A 90/10 split that fits a 16:9 slide above the natural spare strip on a 16:10 display; time and a draggable slider share the strip.
Slide + Time Beside	A 75/25 split for a full-height 4:3 slide, with time and navigation in the side rail.

The names describe fixed geometry. Pulpit does not choose or rearrange a layout based on the deck or display aspect ratio. Built-ins cannot be renamed, overwritten or deleted. **Duplicate to Customize** makes an editable copy and opens it.

Speaker notes

Notes mapping is **explicit and deterministic**. Pulpit does not guess which half of a page is notes, and no heuristic may silently override a mapping you chose. The active mapping is always visible in the presenter window.

Mapping	Meaning
Slides only	PDF page *N* is audience slide *N*. No notes.
Split page	Configured regions of each page are the slide and the notes (`left/right` and `top/bottom` presets ship in the UI).
Paired pages — alternating	Pages alternate slide, notes, slide, notes… (`notes_first` reverses it).
Paired pages — two ranges	The document is slides followed by notes, or the reverse.

Slide indices are logical, so the mapping — not the PDF — decides how many slides the deck has. Changing the mapping advances the render generation and clamps the current slide into the new space.

Where a mapping comes from

In priority order:

1. A mapping you chose for **this document** (`notes.per_document` in `settings.toml`). Nothing overrides it.
2. A recognised **metadata contract** in the PDF, if `notes.honour_metadata_contract` is on.
3. Your **default mapping** (`notes.default_mapping`).

The metadata contract (Typst/Mosaic)

A generator can declare the mapping in any PDF metadata string — `Keywords`, `Subject` or `Title` — as a single whitespace-delimited directive:

```
pulpit:mapping=slides-only
pulpit:mapping=split;slide=0,0,0.5,1;notes=0.5,0,0.5,1
pulpit:mapping=alternating;notes-first=false
pulpit:mapping=two-ranges;notes-first=true
```

Regions are `x,y,width,height` as fractions of the page, origin top-left. A malformed or unknown directive is **rejected**, not approximated: the document then falls back to your default mapping.

In Typst:

```
#set document(keywords: ("pulpit:mapping=alternating;notes-first=false",))
```

The intended pipeline is **Typst/Mosaic → PDF → Pulpit**, while ordinary PDFs keep working without any of this.